Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-213БВ-25

Студент: ИПАЛА Садат

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 03.10.26

Москва, 2026

Постановка задачи

Вариант 9.

В файле записаны команды вида:«число число число». Дочерний процесс производит деление первого числа команда, на последующие числа в команде, а результат выводит в стандартный поток вывода. Если происходит деление на 0, то тогда дочерний и родительский процесс завершают свою работу. Проверка деления на 0 должна осуществляться на стороне дочернего процесса. Числа имеют тип float. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- fork(): pid_t fork(void);создает дочерний процесс
- pipe(): int pipe(int pipefd[2]);создает неименованный канал для передачи данных
- reads(): size_t read(int fd, void *buf, size_t count);читает данные из файлового дескриптора
- write(): ssize_t write(int fd, const void *buf, size_t count);записывает данные в файловый дескриптор
- open(): int open(const char *pathname, int flags, ...);открывает файл
- close(): int close(int fd);закрывает файловый дескриптор
- dup2(): int dup2(int oldfd, int newfd);переназначает файловый дескриптор
- execv(): int execv(const char *pathname, char *const argv[]);заменяет образ памяти процесса
- waitpid(): pid_t waitpid(pid_t pid, int *status, int options);ожидает завершения дочернего процесса
- exit(): void exit(int status);завершает процесс с заданным статусом

Описание работы программы:

Программа реализует межпроцессное взаимодействие между родительским и дочерним процессами через неименованный канал (pipe) в ОС Linux. Родительский и дочерний процессы представлены двумя разными исполняемыми файлами: parent и child.

Родительский процесс первым делом читает из стандартного потока ввода (STDIN_FILENO) имя файла, в котором записаны команды вида «число число число<endline>». После этого родительский процесс создает неименованный канал с помощью системного вызова pipe(). Канал состоит из двух файловых дескрипторов: pipe1[0] — для чтения и pipe1[1] — для записи.

Далее родительский процесс с помощью fork() создает дочерний процесс. В дочернем процессе родительский код заменяется на программу child через системный вызов execv(). Перед этим дочерний процесс выполняет следующие действия:

1.Открывает файл с переданным именем на чтение с помощью open().

2.Переназначает стандартный поток ввода (STDIN_FILENO) на открытый файл с помощью dup2(). Таким образом, дочерний процесс читает команды непосредственно из файла, а не с клавиатуры.

3.Переназначает стандартный поток вывода (STDOUT_FILENO) на дескриптор записи в канал pipe1[1] с помощью dup2(). Все, что дочерний процесс выводит, попадает в канал и становится доступным родительскому процессу.

4.Закрывает неиспользуемые файловые дескрипторы (pipe1[0] и pipe1[1]) с помощью close().

Родительский процесс, в свою очередь, закрывает дескриптор записи в канал pipe1[1], после чего в цикле читает данные из pipe1[0] с помощью read() и выводит их в свой стандартный поток вывода с помощью write(). Таким образом реализуется передача результата работы дочернего процесса пользователю.

Дочерний процесс читает данные из своего стандартного потока ввода (то есть из файла) блоками фиксированного размера (1024 байта) с помощью read(). Каждая прочитанная строка разбивается на отдельные числа с помощью функции strtof(), которая преобразует строковое представление числа в значение типа float. Первое число строки принимается за делимое, а все последующие — за делители. Результат последовательного деления выводится в стандартный поток вывода дочернего процесса (то есть в канал pipe1) с помощью write().

Если в процессе вычислений встречается делитель, равный нулю, дочерний процесс немедленно завершается с кодом ошибки EXIT_FAILURE с помощью системного вызова exit(). Проверка деления на ноль выполняется именно на стороне дочернего процесса, как того требует задание.

Родительский процесс после завершения чтения из канала ожидает завершения дочернего процесса с помощью waitpid() и анализирует его статус. Если дочерний процесс завершился с ненулевым кодом возврата (что означает деление на ноль), родительский процесс также завершает свою работу с кодом ошибки.

Все системные вызовы в программе сопровождаются проверкой на ошибку: при неудачном выполнении программа выводит сообщение об ошибке в стандартный поток ошибок (STDERR_FILENO) и завершается с кодом EXIT_FAILURE.

Код программы

parent.c

```
    ssize_t bytes;
    while ((bytes = read(pipe1[0], buf, sizeof(buf))) > 0) {
        ssize_t written = 0;
        while (written < bytes) {
            ssize_t w = write(STDOUT_FILENO, buf + written, bytes - written);
            if (w <= 0)
                fatal("error: write to stdout failed\n");
            written += w;
        }
```

```
    }
    if (bytes < 0)
        fatal("error: read from pipe failed\n");

    close(pipe1[0]);

    int status;
    if (waitpid(child, &status, 0) == -1)
        fatal("error: waitpid failed\n");

    if (WIFEXITED(status) && WEXITSTATUS(status) != 0)
        fatal("error: child terminated due to division by zero\n");

    return 0;
}
```

child.c

```
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

#define BUF_SIZE 1024
#define MAX_NUMBERS 64

static void fatal(const char *msg) {
    write(STDERR_FILENO, msg, strlen(msg));
    exit(EXIT_FAILURE);
}

static int format_float(float value, char *out, int out_size) {
    int pos = 0;
```

```
int int_part;
float frac;
char tmp[32];
int t;
int k;
int digit;

if (value < 0.0f) {
    if (pos >= out_size - 1) return pos;
    out[pos++] = '-';
    value = -value;
}

int_part = (int)value;

t = 0;
if (int_part == 0) {
    tmp[t++] = '0';
} else {
    while (int_part > 0 && t < 31) {
        tmp[t++] = '0' + (int_part % 10);
        int_part /= 10;
    }
}

for (k = t - 1; k >= 0 && pos < out_size - 1; k--)
    out[pos++] = tmp[k];

if (pos < out_size - 1)
    out[pos++] = '.';
```

```
    frac = value - (float)((int)value);
    for (k = 0; k < 4 && pos < out_size - 1; k++) {
        frac *= 10.0f;
        digit = (int)frac;
        out[pos++] = '0' + digit;
        frac -= (float)digit;
    }

    out[pos] = '\0';
    return pos;
}

int main(void) {
    char buf[BUF_SIZE];
    ssize_t bytes;

    while ((bytes = read(STDIN_FILENO, buf, sizeof(buf) - 1)) > 0) {
        char *cursor;
        char *end;

        buf[bytes] = '\0';
        cursor = buf;
        end = buf + bytes;

        while (cursor < end) {
            char *line_end;
            char *line_start;
            int line_len;
            char saved;

            line_end = cursor;
```

```
while (line_end < end && *line_end != '\n')
    line_end++;

if (line_end < end) {
    saved = *line_end;
    *line_end = '\0';
} else {
    saved = '\0';
}

line_start = cursor;
line_len = (int)(line_end - line_start);

if (line_len > 0) {
    char *saveptr;
    float numbers[MAX_NUMBERS];
    int count;

    saveptr = line_start;
    count = 0;

    while (count < MAX_NUMBERS) {
        char *next;
        float val;

        next = saveptr;
        val = strtof(saveptr, &next);
        if (next == saveptr)
            break;

        numbers[count++] = val;
```

```
    saveptr = next;
}

if (count > 0) {
    float result;
    int error;

    result = numbers[0];
    error = 0;

    for (int k = 1; k < count; k++) {
        if (numbers[k] == 0.0f) {
            error = 1;
            break;
        }
        result /= numbers[k];
    }

    if (error) {
        exit(EXIT_FAILURE);
    }

    {
        char out[64];
        int len;
        ssize_t written;

        len = format_float(result, out, sizeof(out));
        out[len++] = '\n';

        written = 0;
```

```
                        while (written < len) {
                            ssize_t w = write(STDOUT_FILENO, out + written, len
- written);
                            if (w <= 0)
                                fatal("error: child write failed\n");
                            written += w;
                        }
                    }
                }

                if (line_end < end)
                    *line_end = saved;

                cursor = line_end;
                if (cursor < end && *cursor == '\n')
                    cursor++;
            }
        }

        if (bytes < 0)
            fatal("error: child read failed\n");

        return 0;
    }
```

Протокол работы программы

Тестирование:

Тест 1 — Корректные данные без деления на ноль

Данные (файл test1.txt):

10.0 2.0 5.0

100.0 4.0

20.0 2.0 2.0

Ожидаемые:

10.0 / 2.0 / 5.0 = 1.0000

100.0 / 4.0 = 25.0000

20.0 / 2.0 / 2.0 = 5.0000

Проверка:

```
sad@sadat:~$ cd ~/Sdt-OS/lab1
make
./parent
cc -Wall -Wextra -std=c11 -o parent parent.c
cc -Wall -Wextra -std=c11 -o child child.c
test1.txt
1.0000
25.0000
5.0000
sad@sadat:~/Sdt-OS/lab1$
```

Результат совпадает с ожидаемым!

Тест 2 — Деление на ноль

Данные (файл test2.txt):

8.0 2.0

12.0 3.0 2.0

5.0 0.0 2.0

99.0 3.0

Ожидаемые:

8.0 / 2.0 = 4.0000

12.0 / 3.0 / 2.0 = 2.0000

Третья строка содержит делитель 0.0 → завершение дочернего и родительского процессов

Четвёртая строка (99.0 3.0) не обрабатывается

Проверка:

```
sad@sadat:~/Sdt-OS/lab1$ ./parent
test2.txt
```

```
4.0000

2.0000

error: child terminated due to division by zero

sad@sadat:~/Sdt-OS/lab1$
```

Результат совпадает с ожидаемым. Процессы завершены корректно, ошибка обработана.

Тест 3 — Отрицательные числа и десятичные дроби

Данные (файл test3.txt):

-100.0 4.0

7.5 2.5

1.0 3.0

Ожидаемые:

-100.0 / 4.0 = -25.0000

7.5 / 2.5 = 3.0000

1.0 / 3.0 = 0.3333 (округление до 4 знаков после запятой)

Проверка:

```
sad@sadat:~/Sdt-OS/lab1$ ./parent

test3.txt

-25.0000

3.0000

0.3333

sad@sadat:~/Sdt-OS/lab1$ make clean

rm -f parent child

sad@sadat:~/Sdt-OS/lab1$
```

Результат совпадает с ожидаемым. Парсинг отрицательных и дробных чисел работает корректно.

Все проверки корректно выполнены !!!

Вывод

В ходе выполнения лабораторной работы были изучены механизмы межпроцессного взаимодействия в ОС Linux: создание дочерних процессов (fork), замена образа памяти (execv), перенаправление файловых дескрипторов (dup2) и передача данных через неименованные каналы (pipe). Программа реализует разделение первого числа на последующие с обработкой деления на ноль на стороне дочернего процесса. Основной проблемой была необходимость отказаться от функций стандартной библиотеки stdio.h и использовать только системные вызовы, а также

реализовать собственный вывод чисел с плавающей точкой без printf. В целом, работа позволила глубже понять принципы работы процессов и их взаимодействия в ОС Linux.